Three.js VFX to Unity — PRD

Nov 1, 2026 · @Jorge

A personal Unity package (namespace `ConfusedGameDev`) that ports the free Three.js effects from 00GameDevs into URP, each with an editor that exposes the same parameters as the website and previews them live.

Goals and success criteria

v1 succeeds when one effect looks and tunes the same in Unity as on the site, and the next effect takes days, not weeks, because the template is reusable.

1. **Visual parity.** Side-by-side captures of the web effect and the Unity port, with the same parameter values, are indistinguishable at a glance.
2. **Parameter parity.** Every slider, colour, toggle and dropdown on the website exists in the Unity editor with the same name, range and default.
3. **Artist-friendly tuning.** Values change live in Scene view and Play mode, and are saved as reusable presets.
4. **Fast porting.** After the first effect, porting another one takes 2–3 days or less.
5. **Usable in real projects.** Effects drop into Jorge's personal Unity games (for example KISERU, Bosconian VR, Finite Runner) without project-specific code.

Users and use cases

The primary user is Jorge as programmer and effect tuner; a secondary user is anyone he shares the package with later.

User	Use case	What they need
Programmer (Jorge)	Port a new effect from the site	A template, porting rules and a checklist so each port follows the same structure
Programmer (Jorge)	Trigger effects from gameplay	A small C# API: `Play()`, `Stop()`, `SetPreset()`, events on finish
Tuner / artist	Match or restyle an effect	The website's controls in the Inspector, live preview, presets to save and swap

User	Use case	What they need
Tuner / artist	Copy a look from the site	Paste the site's parameter values into a preset in one step

Scope

v1 ports a pilot set of effects by hand against a shared template; automatic conversion is out of scope.

In scope (v1)

- Pilot effects: fireball, energy burst and magic barrier (they cover a projectile, a one-shot burst and a looping shell).
- URP shaders (HLSL, with Shader Graph Custom Function wrappers where useful).
- Runtime components for spawning, animation, particles and lifetime.
- Preset ScriptableObjects mirroring the website's parameters.
- Custom Inspector plus an Effect Lab EditorWindow with live preview.
- Import of parameter values copied from the site (JSON or the copied JS config).
- Distribution as a UPM package from a Git repo.

Out of scope (v1)

- Automatic GLSL-to-HLSL or JS-to-C# conversion of arbitrary effects.
- Built-in Render Pipeline and HDRP.
- Porting the website's grass, terrain or object assets (effects first).
- Netcode sync of effects (games handle that themselves).
- An Asset Store release.

Source material

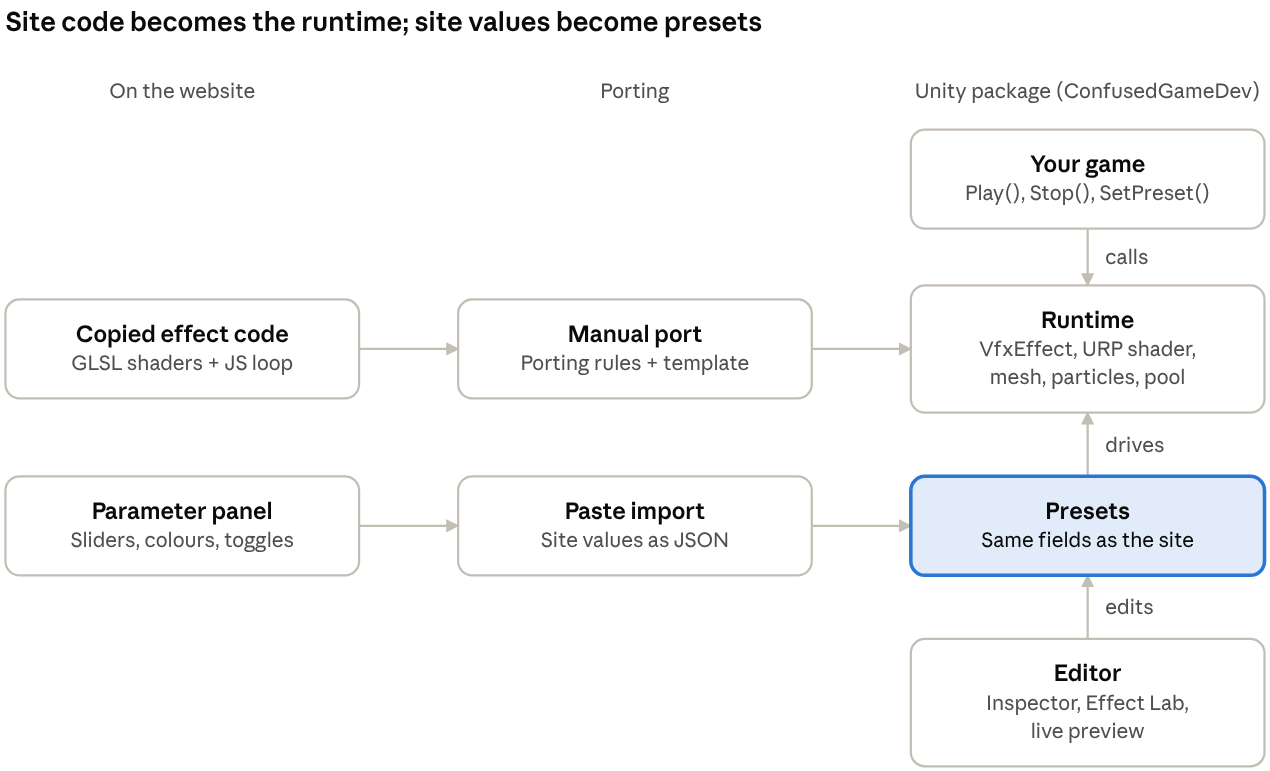
00GameDevs is a free Three.js effects library with live preview, adjustable parameters and one-click copy; the site states that exported assets may be used commercially and modified (00GameDevs). Its showcase includes fireball, ice shard, prism guard, fire vortex, painterly grass, magic barrier, arcane matrix, energy burst and tidal surge.

The page renders in JavaScript, so the exact parameter lists and copied-code format were not visible when this PRD was written. A typical Three.js effect, and therefore what each port must cover, has five parts:

Part on the web	Typical form	Unity counterpart
Shader	`ShaderMaterial` with GLSL vertex/fragment and uniforms	URP HLSL shader, uniforms as material properties
Geometry	Built-in or procedural `BufferGeometry` (spheres, rings, shards, planes)	Procedural mesh generated in C#, or baked mesh asset
Particles	`Points` or instanced meshes updated in JS	VFX Graph (PC) or Particle System / GPU instancing (Quest)
Animation	Time-driven updates in the render loop	C# component driving `_Time` offsets and parameter curves
Controls	GUI panel (lil-gui style) bound to uniforms	Preset ScriptableObject + custom Inspector

Milestone 0 confirms this table against the copied code of the three pilot effects.

Architecture



architecture · web inputs to Unity package

The copied code is ported by hand into the runtime; the website's control values flow into presets, which drive the runtime and are tuned in the editor.

Functional requirements

Every effect ships as the same four pieces: a shader, a runtime component, a preset type and an editor, all generated from one template.

ID	Area	Requirement	Priority
FR-01	Runtime	Each effect is a prefab with one `VfxEffect` component (base class) that exposes `Play()`, `Stop(bool immediate)`, `SetPreset(VfxPreset preset)` and an `onFinished` event.	Must
FR-02	Runtime	Effects run looping or one-shot, set per preset, with optional auto-destroy or return to pool.	Must

ID	Area	Requirement	Priority
FR-03	Runtime	Parameters can be changed at runtime from code and animated (for example charge-up), without editing the shared preset asset.	Must
FR-04	Runtime	Effects use MaterialPropertyBlocks or per-instance materials so several instances with different values render correctly.	Must
FR-05	Runtime	A simple pool (`VfxPool`) spawns and recycles effects to avoid GC spikes.	Should
FR-06	Presets	Each effect has a `VfxPreset` ScriptableObject subclass whose fields match the website's controls: same labels, ranges (`[Range]`), defaults, colours and enums.	Must
FR-07	Presets	Field order and grouping follow the website's panel so the two can be compared row by row.	Must
FR-08	Presets	Paste import: values copied from the site (JSON or JS config object) fill a preset, with unknown keys reported, not dropped silently.	Should
FR-09	Presets	Export a preset back to JSON for diffing and sharing.	Could
FR-10	Editor	Custom Inspector shows the preset with the website's grouping, live updates in Scene view while dragging, and Play / Stop / Replay buttons that work outside Play mode.	Must
FR-11	Editor	Effect Lab EditorWindow: pick an effect and preset, preview it in an isolated stage with dark background, orbit camera, timeline scrub and play speed.	Should
FR-12	Editor	Reset-to-website-defaults and randomize buttons per preset.	Should
FR-13	Editor	Side-by-side reference: load a screenshot or short video of the web effect beside the preview for parity checks.	Could

ID	Area	Requirement	Priority
FR-14	Tooling	A template generator (menu item) creates the folder, shader stub, component, preset and editor for a new effect from its name and parameter list.	Should
FR-15	Samples	A sample scene shows every ported effect with its default preset and one variant.	Must

Porting rules

Most of a port is mechanical; the rules below are where ports usually go wrong and are checked in every review.

Three.js / GLSL	Unity URP / HLSL	Watch out for
`vec2/3/4`, `mat3/4`	`float2/3/4`, `float3x3/4x4`	Matrix multiply order: GLSL `M * v` is `mul(M, v)`
`mix`, `fract`, `mod`	`lerp`, `frac`, custom `mod`	HLSL `fmod` keeps the sign of x; write `x - y * floor(x / y)` to match GLSL `mod`
`texture2D` / `texture`	`SAMPLE_TEXTURE2D(tex, sampler, uv)`	UV origin and texture wrap/filter settings
`uTime` uniform	`_Time.y`, or a per-instance start-time property	One-shot effects need their own clock so replays restart at 0
`uniform` values	Material properties in `CBUFFER_START(UnityPerMaterial)`	Keep SRP Batcher compatible
`transparent`, `depthWrite: false`	`Queue=Transparent`, `ZWrite Off`	Sort order between layers of the same effect

Three.js / GLSL	Unity URP / HLSL	Watch out for
`AdditiveBlending`	`Blend One One` (or `SrcAlpha One`)	Premultiplied vs straight alpha
`side: DoubleSide`	`Cull Off`	Normals on back faces
Colours in sRGB	Linear colour space	Convert defaults so colours match the site
Three.js units, Y-up, right-handed	Unity metres, Y-up, left-handed	Mirror Z in procedural geometry and directions
`requestAnimationFrame` update	`Update()` with `Time.deltaTime`	Frame-rate independence; no per-frame allocations
Post-processing passes (bloom etc.)	URP Volume overrides, or a Renderer Feature (Render Graph)	Only when an effect depends on its own pass

Non-functional requirements

Effects target PC at full quality and Meta Quest / PICO with a lighter tier, and follow Jorge's code standards.

ID	Requirement
NFR-01	Engine: Unity 6 with URP (exact version set in Milestone 0).
NFR-02	PC: each effect costs under 0.5 ms GPU at 1080p on a mid-range GPU.
NFR-03	Quest / PICO: each effect has a Mobile quality tier that holds 72 fps in a test scene with 5 instances; single-pass instanced stereo supported.

ID	Requirement
NFR-04	No per-frame GC allocations in runtime code.
NFR-05	Shaders are SRP Batcher compatible and compile for DX11/12, Vulkan and GLES3.
NFR-06	All code lives in the `ConfusedGameDev` namespace (for example `ConfusedGameDev.Vfx`, `ConfusedGameDev.Vfx.Editor`), with separate runtime and editor assemblies.
NFR-07	C# parameters use lowerCamelCase.
NFR-08	Package installs through UPM from a Git URL, with no dependency beyond URP (VFX Graph optional, behind a define).
NFR-09	Each effect folder includes a short README: source effect, link, parameter list and known differences from the web version.

Milestones

The plan runs in order; each milestone ends with a parity check against the website before the next starts. Durations are rough estimates for evening/weekend work.

1. **M0 — Discovery (2–3 days).** Copy the code of the three pilot effects, list every parameter and range, confirm the Source material table, fix the Unity version.
2. **M1 — Framework (1 week).** Package skeleton, assemblies, `VfxEffect` base, `VfxPreset` base, pool, Inspector base with live preview and Play/Stop/Replay.
3. **M2 — Pilot: Fireball (1 week).** Shader, mesh, particles and preset; parity screenshots; porting rules updated with what was learned.
4. **M3 — Energy burst and magic barrier (1–2 weeks).** Prove the template on a one-shot and a looping effect.
5. **M4 — Editor tooling (1 week).** Effect Lab window, paste import, reset/randomize, template generator.
6. **M5 — Quest tier (1 week).** Mobile quality tier for the pilot effects, profiled on device.
7. **M6 — Library rollout (ongoing).** Remaining effects, one at a time, each under 2–3 days.

Risks and mitigations

The main risk is effects that depend on Three.js specifics (post-processing, blending, colour space) and stop matching once moved.

Risk	Impact	Mitigation
Colours or glow look different (linear vs sRGB, tonemapping, bloom)	Parity fails though the code is right	Match colour space and tonemapping in the sample scene first; convert defaults; parity check in M2
Copied code is minified or depends on site-internal helpers	Slow or incomplete ports	M0 inspects the copied code before committing to a pilot; drop effects that are not portable
Effect needs its own render pass	Not portable as a plain material	Renderer Feature with Render Graph, only when necessary; otherwise approximate with Volume bloom
Heavy particle counts on Quest	Frame drops in VR	Mobile tier with lower counts and simpler shaders (M5)
Website changes its effects or parameters	Presets no longer match	Record the source date and copied code per effect in its README
License terms change	Usage restrictions	Save the license statement with the date of each port

Open questions

- [ ] Project name and package id (working assumption: `com.confusedgamedev.vfx`).
- [ ] Unity version: Unity 6.0 LTS or a newer Unity 6 release?
- [ ] Pilot effects: are fireball, energy burst and magic barrier the right first three?
- [ ] Is Quest/PICO support needed in v1, or can the Mobile tier wait until a game needs it?
- [ ] VFX Graph or Particle System as the default particle backend?
- [ ] Public repo (shareable) or private?

- [ ] Should the site's non-effect assets (grass, objects) be a later phase?

Sources

- 00GameDevs effects library (zh-TW) — effect list, live tuning and commercial-use statement.